

Tuần 2 — **Lambda** nâng cao (versions, aliases, layers, concurrency, event sources)

Domain: Domain 1 – Development (32%) · **Thời lượng:** ~11h (4 buổi) · **Vị trí:** Tuần 2/5 của Domain 1

Điều hướng:  [Tuần 1](#) ·  [Kế hoạch tổng](#) · [Tuần 3](#) 

Mục tiêu tuần này

- Tự tay publish một **version**, tạo **alias**, và cấu hình weighted routing (canary 90/10) giữa 2 version.
- Giải thích được khác biệt giữa **reserved concurrency** và **provisioned concurrency**, và biết khi nào dùng cái nào.
- Nói rõ được luồng xử lý lỗi của **async invoke**: retry mấy lần → đi đâu (DLQ vs destinations).
- Tự tạo được **event source mapping** cho **SQS** → **Lambda** và **DynamoDB Streams** → **Lambda**.
- Giải thích được **cold start** là gì và liệt kê được các cách giảm.
- Nhớ chính xác các giới hạn số của **Lambda** hay bị hỏi trong đề (payload, package size, **/tmp**, layers, env vars).

Nội dung học chi tiết

Buổi A — Lý thuyết (~3h)

1. **\$LATEST** vs versions (tính bất biến)

- Mỗi lần deploy code mới → cập nhật vào **\$LATEST** (mutable, luôn thay đổi).
- Khi **publish version** → chụp lại snapshot code + config tại thời điểm đó → **bất biến** (immutable), không sửa được nữa.

- Mỗi version có **ARN riêng** kèm số version (vd `...:function:myFn:3`); **\$LATEST** là con trỏ tới bản đang phát triển.
- Vì sao quan trọng: để hay hỏi "làm sao đảm bảo code chạy production không bị thay đổi ngoài ý muốn" → **publish version**.

2. Aliases + weighted routing (canary)

- **alias** = con trỏ có tên (vd **prod**, **dev**) trỏ tới **1 version cụ thể**.
- Ứng dụng gọi qua alias ARN → muốn đổi version chỉ cần trỏ lại alias, client không cần đổi ARN.
- **Weighted routing**: 1 alias có thể chia traffic giữa **đúng 2 version** (vd 90% version cũ, 10% version mới) → dùng làm **canary / blue-green** deploy.
- **! Alias KHÔNG trỏ tới alias khác** — chỉ trỏ tới version.

3. Layers + environment variables

- **Layer** = gói thư viện/dependency dùng chung, tách khỏi code function → tái sử dụng, giảm dung lượng deploy package.
- Giới hạn: **tối đa 5 layer/function**; tổng dung lượng **giải nén** (function code + layers) **≤ 250 MB**.
- **Environment variables**: tổng **≤ 4 KB**; có thể **mã hoá bằng KMS** (dùng key riêng cho biến nhạy cảm như secret/connection string).

4. Concurrency: reserved vs provisioned

- **Concurrency mặc định** mỗi account/region: **1000** (soft limit, xin tăng được).
- **Reserved concurrency**: đặt **trần** cho 1 function + **đảm bảo** phần concurrency riêng cho nó. Đồng thời **giới hạn ảnh hưởng** lên các function khác (vì phần đã reserve bị trừ khỏi pool chung). Không tốn thêm phí.
- **Provisioned concurrency**: **giữ sẵn** số môi trường **đã khởi tạo ấm sẵn** → loại bỏ **cold start**. Có tính phí cho phần giữ sẵn.
- Vượt concurrency → bị **throttle** → trả **`TooManyRequestsException`** (HTTP 429).

5. Cold start

- Là độ trễ lần gọi đầu khi **Lambda** phải: tải code → khởi tạo runtime → chạy code init ngoài handler.
- Cách giảm: dùng **provisioned concurrency**; giảm dung lượng package; đưa việc nặng ra ngoài handler (init 1 lần); ngôn ngữ khởi động nhanh; tránh VPC config thừa.

- **SnapStart** — giảm cold start (best-effort) cho **Java 11+**, **Python 3.12+**, **.NET 8+**; chỉ dùng trên **published version/alias** (không **\$LATEST**); **không dùng chung provisioned concurrency**; không hỗ trợ **/tmp > 512 MB**.

B Buổi B — Hands-on (~3.5h)

 **Lab cầm tay chỉ việc (từng bước + lệnh + code):** labs.md.

Thay **myFn** bằng tên function của bạn.

Bước 1 — Publish version từ \$LATEST:

```
aws lambda publish-version --function-name myFn --description "v1 stable"
```

Bước 2 — Tạo alias **prod trỏ tới version 1:**

```
aws lambda create-alias --function-name myFn --name prod --function-version 1
```

Bước 3 — Publish version 2 rồi cấu hình weighted alias canary 90/10 (prod chính = v1 giữ 90%, v2 nhận 10%):

```
aws lambda publish-version --function-name myFn --description "v2 canary"
aws lambda update-alias --function-name myFn --name prod \
  --function-version 1 \
  --routing-config '{"AdditionalVersionWeights":{"2":0.10}}'
```

Bước 4 — Tạo và gắn layer:

```
# đóng gói dependency vào layer.zip (thư mục nodejs/node_modules/)
aws lambda publish-layer-version --layer-name my-deps \
  --zip-file fileb://layer.zip --compatible-runtimes nodejs24.x
# gắn layer vào function
aws lambda update-function-configuration --function-name myFn \
  --layers <layer-version-arn>
```

Bước 5 — Set reserved và provisioned concurrency:

```
# đặt trần + đảm bảo phần riêng
aws lambda put-function-concurrency --function-name myFn --reserved-
concurrent-executions 50
# giữ sẵn môi trường ảo (gắn vào alias/version, không dùng $LATEST)
aws lambda put-provisioned-concurrency-config --function-name myFn \
--qualifier prod --provisioned-concurrent-executions 5
```

Bước 6 — Async invoke + cấu hình DLQ (SQS) và destinations:

```
# gọi kiểu async (Event)
aws lambda invoke --function-name myFn --invocation-type Event \
--payload '{"k":"v"}' out.json
# DLQ dùng SQS
aws lambda update-function-configuration --function-name myFn \
--dead-letter-config '{"TargetArn":"arn:aws:sqs:...:my-dlq"}'
# destinations: onSuccess + onFailure
aws lambda put-function-event-invoke-config --function-name myFn \
--destination-config '{"OnSuccess":
{"Destination":"arn:aws:sqs:...:ok"},"OnFailure":
{"Destination":"arn:aws:sqs:...:fail"}}'
```

Bước 7 — Event source mapping (Lambda poll nguồn):

```
# SQS -> Lambda
aws lambda create-event-source-mapping --function-name myFn \
--event-source-arn arn:aws:sqs:...:my-queue --batch-size 10
# DynamoDB Streams -> Lambda
aws lambda create-event-source-mapping --function-name myFn \
--event-source-arn <dynamodb-stream-arn> --starting-position LATEST
```

C Buổi C — Bổ sung (~2.5h)

1. Invocation model — nắm chắc 3 kiểu:

- **Sync (RequestResponse)**: gọi và chờ kết quả (vd **API Gateway**, CLI mặc định). Payload tối đa **6 MB**. Lỗi → trả về ngay cho caller, caller tự retry.
- **Async (Event)**: **Lambda** xếp vào internal queue rồi trả **202** ngay. Payload tối đa **1 MB** (AWS đã nâng từ 256 KB — đề/khoá cũ có thể vẫn ghi 256 KB). Lỗi → **Lambda** tự retry.
- **Event source mapping (poll)**: **Lambda** chủ động poll từ **SQS / Kinesis / DynamoDB Streams** rồi gọi function theo batch.

2. Xử lý lỗi async (RẤT hay hỏi):

- Thất bại → retry **2 lần** (tổng cộng **3 lần thử**).
- Sau khi hết retry → gửi tới **DLQ** (**SQS** hoặc **SNS**) hoặc **destinations OnFailure** (**SQS** / **SNS** / **Lambda** / **EventBridge**).
- **Destinations** hiện đại hơn DLQ: hỗ trợ cả **OnSuccess** lẫn **OnFailure** và kèm nhiều metadata hơn.

3. /tmp ephemeral storage:

- Mặc định **512 MB**, cấu hình được tới **10 GB**.
- **KHÔNG bền** giữa các lần invoke (chỉ tồn tại trong vòng đời execution environment) → cần lưu bền phải dùng **S3** / **DynamoDB** / **EFS**.

4. Đóng gói & VPC:

- **Deployment package: 50 MB** (zip upload trực tiếp qua API), **250 MB** (giải nén, đã gồm layers), **10 GB** (container image).
- Zip vs container image: image hợp khi package lớn / cần custom runtime.
- **Lambda trong VPC**: đặt vào private subnet để truy cập tài nguyên nội bộ (RDS...); muốn **ra internet** phải qua **NAT Gateway** (subnet công khai không cấp IP public cho ENI của Lambda).

Đọc thêm: **Lambda** Developer Guide — mục *Versions, Aliases, Concurrency, Async invocation, Event source mappings*.

Buổi D — Practice + Review (~2h)

 **Bộ câu hỏi luyện tập của tuần:** [questions.md](#) — đáp án & giải thích: [answers.md](#). (bằng tiếng Anh — văn phong để thật để làm quen để.)

- Làm 20–30 câu practice chủ đề **Lambda** (versions/aliases/concurrency/invoke/event source).
- Ghi **số câu sai**: mỗi câu sai ghi lại keyword + lý do chọn nhầm.
- Ôn **spaced repetition**: xem lại flashcard số liệu ở mốc **1 / 3 / 7 ngày**.
- Tự đọc to phần "Công tự kiểm tra" bên dưới, trả lời không nhìn tài liệu.



PHẢI NHỚ tuần này

Chủ đề	Con số / Sự thật cốt
Versions	Bất biến, có ARN riêng theo số ; \$LATEST là mutable
Alias	Trở tới 1 version ; weighted routing giữa 2 version ; KHÔNG trở alias→alias
Layers	Tối đa 5 layer/function ; tổng giải nén (function + layers) $\leq$ 250 MB
Env vars	Tổng $\leq$ 4 KB ; mã hoá được bằng KMS
Concurrency mặc định	1000 /account/region (soft limit)
Concurrency scaling	1.000 execution environments / 10 giây / function
Throttle	Vượt $\rightarrow$ TooManyRequestsException (HTTP 429)
Reserved concurrency	Đặt trần + đảm bảo phần riêng; giới hạn ảnh hưởng function khác
Provisioned concurrency	Giữ sẵn môi trường ấm $\rightarrow$ loại bỏ cold start (có phí)
SnapStart	Java 11+/Python 3.12+/.NET 8+; chỉ version+alias; best-effort; $\neq$ provisioned concurrency
Payload sync	6 MB (RequestResponse)
Response streaming	tới 200 MB (cách vượt cap 6 MB của sync)
Payload async	1 MB (Event) — <i>AWS đã nâng từ 256 KB</i>
Async retry	Retry 2 lần (tổng 3 lần thử) $\rightarrow$ DLQ hoặc destinations onFailure
DLQ đích	SQS / SNS (chỉ onFailure)
Destinations đích	SQS / SNS / Lambda / EventBridge (cả onSuccess + onFailure)
/tmp	512 MB mặc định, tới 10 GB ; không bền giữa các invoke
Deployment package	50 MB zip upload · 250 MB giải nén · 10 GB container image

⚠ Bẫy đề hay gặp

- Thấy "đảm bảo function có sẵn capacity, không bị function khác chiếm" → dễ chọn provisioned, nhưng nếu chỉ cần **đảm bảo/giới hạn slot** (không nói cold start) thì đúng là **reserved concurrency**.
- Thấy "loại bỏ / giảm **cold start** cho traffic ổn định" → đúng là **provisioned concurrency**, KHÔNG phải reserved.
- Thấy "giảm **cold start**" → có 2 lựa chọn: **provisioned concurrency** (đảm bảo, có phí giữ ấm) hoặc **SnapStart** (Java/Python/.NET, miễn phí, best-effort, chỉ version/alias) — chọn theo runtime & chi phí.
- Thấy "trở alias **prod** sang một alias **blue** khác" → **sai**, alias chỉ trở tới **version**.
- Thấy "canary 10% sang code mới" → dùng **weighted alias** giữa 2 version, KHÔNG phải deploy 2 function.
- Thấy "cần lưu file tạm lớn giữa các bước xử lý trong 1 invoke" → **/tmp** OK; nhưng "giữ state **bền** giữa các invoke" → phải **S3/DynamoDB/EFS**.
- Thấy "async invoke thất bại đi đâu" → sau **3 lần thử** vào **DLQ hoặc destinations**, KHÔNG mất im lặng ngay lần đầu.
- Thấy "**Lambda** trong VPC gọi API internet lỗi timeout" → thiếu **NAT Gateway** (private subnet không tự ra internet).
- Thấy "payload 5 MB gọi trực tiếp OK, 5 MB async lỗi" → async giới hạn **1 MB**, sync mới tới **6 MB**.

↺ Phản xạ nhanh (keyword → đáp án)

Thấy từ khoá	Bật ngay
"immutable snapshot của code"	publish version
"con trở tới version, đổi mà không đổi ARN client"	alias
"canary / chia 10% traffic sang bản mới"	weighted alias (2 version)
"loại bỏ cold start / giữ môi trường ấm"	provisioned concurrency
"giảm cold start cho Java/Python/.NET, không muốn trả phí giữ ấm liên tục"	SnapStart
"đảm bảo slot riêng / giới hạn function ngốn hết"	reserved concurrency
"HTTP 429 / TooManyRequestsException"	bị throttle (vượt concurrency)

Thấy từ khoá	Bật ngay
"async invoke lỗi lưu lại để xử lý sau"	DLQ hoặc destinations OnFailure
"cả xử lý thành công lẫn thất bại theo route"	Lambda destinations (OnSuccess/OnFailure)
"Lambda đọc SQS/Kinesis/DynamoDB Streams "	event source mapping (poll)
"chia sẻ dependency giữa nhiều function"	Layer
"mã hoá biến môi trường nhạy cảm"	env var + KMS
"cần dung lượng đĩa tạm lớn khi chạy"	tăng /tmp (tới 10 GB)
"Lambda trong VPC cần gọi internet"	NAT Gateway
"package > 250 MB / cần custom runtime"	container image (10 GB)



Lab checklist

- ☐ Publish version từ **\$LATEST** và xem ARN có số version
- ☐ Tạo alias **prod** trỏ tới version 1
- ☐ Cấu hình weighted alias canary **90/10** giữa 2 version
- ☐ Tạo layer và gắn vào function
- ☐ Set **reserved concurrency**
- ☐ Set **provisioned concurrency** (gắn vào alias)
- ☐ Async invoke (**--invocation-type Event**) và cấu hình **DLQ (SQS)**
- ☐ Cấu hình **destinations** OnSuccess + OnFailure
- ☐ Tạo **event source mapping** **SQS** → **Lambda**
- ☐ Tạo **event source mapping** **DynamoDB Streams** → **Lambda**



Cổng tự kiểm tra (phải trả lời trôi chảy mới sang tuần sau)

- Reserved vs provisioned concurrency khác gì? Đáp án gọn:** Reserved = đặt trần + đảm bảo phần concurrency riêng cho function (và giới hạn ảnh hưởng

function khác), không tự lo cold start, không thêm phí. Provisioned = giữ sẵn số môi trường đã khởi tạo ấm → loại bỏ cold start, có tính phí.

2. **Async invoke lỗi thì retry mấy lần và đi đâu? Đáp án gọn:** Retry **2 lần** (tổng **3 lần thử**); hết thì đẩy vào **DLQ (SQS/SNS)** hoặc **destinations OnFailure (SQS/SNS/Lambda/EventBridge)**.
3. **Alias có trở tới alias khác được không? Đáp án gọn: Không.** Alias chỉ trở tới **version** (và có thể weighted giữa 2 version).
4. **Cold start là gì, giảm bằng cách nào? Đáp án gọn:** Độ trễ lần gọi đầu do phải tải code + khởi tạo runtime + chạy init. Giảm bằng **provisioned concurrency**, giảm dung lượng package, đưa init nặng ra ngoài handler, chọn runtime khởi động nhanh.
5. **Vì sao KHÔNG lưu state bền vào /tmp? Đáp án gọn:** **/tmp** (512 MB, tới 10 GB) chỉ tồn tại trong vòng đời execution environment, **không bền** giữa các invoke → dữ liệu bền phải dùng **S3/DynamoDB/EFS**.
6. **Giới hạn payload sync và async là bao nhiêu? Đáp án gọn:** Sync (**RequestResponse**) **6 MB**; async (**Event**) **1 MB** (AWS đã nâng từ 256 KB).



Tài nguyên tuần này

 **Đã crawl sẵn tài liệu AWS vào [resources/](#)** — đọc offline được.

- AWS Docs: **Lambda** Developer Guide — mục *Lambda function versions*, *Lambda function aliases*, *Configuring reserved/provisioned concurrency*, *Asynchronous invocation*, *Lambda event source mappings*.
- AWS Docs: **Lambda** Developer Guide — *Using layers*, *Configuring ephemeral storage (/tmp)*, *Configuring VPC access*.
- AWS FAQ: **AWS Lambda** FAQs (mục limits & pricing).
- Stephane Maarek — section **AWS Lambda** (versions/aliases, concurrency, event source mapping).
- Adrian Cantrill — phần Serverless / **Lambda** advanced.



Checklist hoàn thành Tuần 2

- ☐ Hoàn thành 4 buổi A/B/C/D
- ☐ Làm xong toàn bộ Lab checklist (10 mục)
- ☐ Thuộc bảng "PHẢI NHỚ" (không nhìn vẫn đọc đúng số)
- ☐ Làm ≥ 20 câu practice **Lambda** và ghi sổ câu sai
- ☐ Vượt Cổng tự kiểm tra (6/6 câu trả lời trôi chảy)